CMPS 4121 - Advanced Web Tech - Test 1 Checkup 1 Writeup

ImageLab -- Version 1

Reynerio Samos - 2018119235

CMPS 4191 - Advanced Web Technologies - Test 1

Async 202 Accepted + Short Polling

Assessment: Build a synchronous image processing application where an API accpets work, a background worker performs processing on the image and the browser observes the job state.

Work done:

- Landing page for application
- Database schema and migrations
- http handlers and models for different payloads made
- skeleton code migrated from gatekeeper async code
- basic image upload to DB
- rudimentary image filetype check.

Current status: Week 1

Check-in	Primary Focus	Expectation	Date	Pass?	Work needed	
Week 1	Contract, setup, schema, upload and original storage	Application starts; database and upload foundation work	9/9/2026	Maybe	Make a disclaimer for an accepted image type, Change the UI to look more like the	

Check-in	Primary Focus	Expectation	Date	Pass?	Work needed	
					slides, add validation message when user uses bad file type, change ids in DB from bigserial -> UUID	
Week 2						

Week 1

Contract, Requirements and User Flow

Contract Description

Image lab:

- Is a single page application (SPA)
- Preserves and Exposes state through PostgreSQL

When a user selects and submits an image:

- They receive a 202 Accepted
- Later view generated image varients

Image Processing must:

- NOT run inside upload request

Contract Flowchart

Display Mermaid diagrams in this vault?

Only allow if you trust this vault's contents.

Allow

```
graph TD
    A["IMAGE LAB"]

    B["User Uploads Image"]

    C{Validation Check<br/>JPEG/PNG only<br/>Max 10 MB}

    D["Valid Image<br/>Response: 202 Accepted"]

    E[" Invalid File<br/>Response: 400 Bad Request<br/>or 415 Unsupported Media Type"]

    F["Image Stored<br/>in Database"]

    G["View Generated<br/>Image Variants"]

    H["Processing<br/>NOT run inside<br/>uploaded request"]

    A --> B
    B --> C
    C -->|Pass| D
    C -->|Fail| E
    D --> F
    F --> H
    H --> G
```

API Contract: POST /v1/images

Aspect	Rule / Specification
Method & Path	`POST /v1/images`
Content-Type	`multipart/form-data` (Single image file)
Validation Rules	Server must accept JPEG/PNG. Server must enforce maximum size of 10 MB.

Aspect	Rule / Specification
Success Response	`202 Accepted`
Rejection Response	`400 Bad Request` or `415 Unsupported Media Type` (No job created)

Requirements (Only Focusing on WeeK 1 Requirements)

1. Core Architecture & Scope

- ARC-01: Backend must be a Go HTTP API with one in-process background worker.
- ARC-02: Database must be PostgreSQL for images, jobs, and variant metadata.
- ARC-03: File storage must be the local filesystem for original and generated images.
- ARC-04: The core pattern is 202 Accepted + Persistent Job + One Worker + Short Polling.
- ARC-05: Image processing MUST NOT run inside the HTTP upload request.

2. API Contract (HTTP & JSON)

- API-01: Endpoint `POST /v1/images` receives one image via multipart/form-data.
- API-02: Before returning 202, the handler must: validate upload → store original → create image record → create queued job.
- API-03: The request handler must not generate variants.
- API-04: Success must return HTTP 202 Accepted with:
  - Location: `/v1/jobs/{job_id} header.`
  - JSON body: `{ "image_id": 108, "job_id": 42, "status": "queued", "status_url": "/v1/jobs/42" }`
- API-05: `GET /v1/jobs/{job_id}` returns current job state and completed variant metadata.

- API-06: `GET /v1/images/{image_id}/variants/{name}` returns one known generated variant

4. Upload Validation & Storage (Week 1)

- VAL-01: Accept JPEG and PNG images no larger than 10 MB.
- VAL-02: Server must enforce type, size, presence, and decodability.
- VAL-03: Generate server-controlled stored filenames. Never use the submitted filename directly as a filesystem path.
- VAL-04: Original filename may be stored as display metadata only.
- VAL-05: If validation, storage, or record creation fails, do not return 202 and do not claim a job exists.

User Flow


```mermaid
sequenceDiagram
    autonumber

    actor User
    participant Browser
    participant API
    participant Storage
    participant PostgreSQL
    participant Worker

    User->>Browser: Select JPEG or PNG
    Browser->>Browser: Preview image

    User->>Browser: Click Process image
    Browser->>API: POST image
    activate API

    API->>API: Validate image
```

```
    API->>Storage: Store original image
    Storage-->>API: Original image stored

    API->>PostgreSQL: Create job with status queued
    PostgreSQL-->>API: image_id, job_id

    API-->>Browser: 202 Accepted<br/>image ID, job ID, status URL
    deactivate API

    Note over Worker, PostgreSQL: Worker claims job immediately in
background
    Worker->>PostgreSQL: Claim queued job
    PostgreSQL-->>Worker: Job claimed
    Worker->>PostgreSQL: Set status to processing
    Worker->>Worker: Generate required variants (may take time)
    Worker->>Storage: Store generated variants
    Worker->>PostgreSQL: Set status to completed or failed

    loop Poll status every second
        Browser->>API: GET status URL
        API->>PostgreSQL: Read job status
        PostgreSQL-->>API: Current status
        API-->>Browser: Status response
        Browser->>Browser: Update job card

        alt Status is queued
            Note over Browser,Worker: Continue polling
        else Status is processing
            Note over Browser,Worker: Worker is generating
variants
        else Status is completed
            Browser->>Browser: Stop polling
            Browser->>Browser: Show results
        else Status is failed
            Browser->>Browser: Stop polling
            Browser->>Browser: Show failure state
        end
    end
```

Database Schema

Display Mermaid diagrams in this vault?
Only allow if you trust this vault's contents.

Allow

```
erDiagram
    IMAGES ||--o{ JOBS : "prepares"
    IMAGES ||--o{ VARIANTS : "prepares"

    IMAGES {
        bigint id PK
        text original_filename "Metadata only"
        text stored_filename "Server-generated, unique"
        text media_type "Validated"
        bigint size_bytes "Validated"
        timestamptz created_at
    }

    JOBS {
        bigint id PK
        bigint image_id FK
        job_status status "queued|processing|completed|failed"
        timestamptz queued_at
        timestamptz started_at "Nullable (Week 2)"
        timestamptz completed_at "Nullable (Week 2)"
        timestamptz failed_at "Nullable (Week 2)"
    }

    VARIANTS {
        bigint id PK
        bigint image_id FK
        text name "thumbnail|preview|display"
        text stored_filename
        integer width
        integer height
        bigint size_bytes
    }
```

Main Architecture Diagram

Display Mermaid diagrams in this vault?

Only allow if you trust this vault's contents.

Allow

```mermaid
flowchart LR

    subgraph Browser["Client (Browser SPA)"]
        UI["Select & Submit Image<br/>Poll Job Status"]
    end

    subgraph API["Go HTTP Handlers"]
        POST["POST /v1/images<br/>Validate & Store"]
        GET["GET /v1/jobs/{id}<br/>Read State"]
    end

    subgraph Worker["Background Worker"]
        W["Process Job & Generate Variants"]
    end

    subgraph DB["PostgreSQL Database"]
        IMG[("images table")]
        JOB[("jobs table")]
        VAR[("variants table")]
    end

    FS[("Local Filesystem<br/>Original & Variants")]

    UI -->|"1. Upload Image"| POST
    POST -->|"2. Store Original"| FS
    POST -->|"3. Create Image + Job"| IMG
    POST -->|"4. Create Job"| JOB
    POST -->|"5. Return 202"| UI

    W -->|"6. Claim Job"| JOB
    W -->|"7. Read Original"| FS
    W -->|"8. Generate + Write Variants"| FS
    W -->|"9. Store Metadata + Mark Complete"| VAR
    W -->|"10. Update Job Status"| JOB
```

```
    UI -->|"11. Poll (Every 1s)"| GET
    GET -->|"12. Read Job State"| JOB
    GET -->|"13. Read Variants"| VAR
    GET -->|"14. Return State"| UI

    class UI client;
    class POST,GET api;
    class W worker;
    class IMG,JOB,VAR database;
    class FS storage;
```

Component Responsibilities

Component	Responsibilities
Browser (SPA)	Select & preview image, send one POST request, short-poll `GET /v1/jobs/{id}` every 1 second, render results/failures.
HTTP API	Validate (JPEG/PNG, ≤10MB), generate safe stored filename, write original to FS, create `images` + `jobs` (queued) records, return `202 Accepted` with status URL.
Background Worker	Claim `queued` job via transaction + row locking, read original from FS, generate `thumbnail`, `preview`, `display` variants, write files, insert `variants` metadata, mark job `completed` or `failed`.
PostgreSQL	Authoritative state: `images` (metadata), `jobs` (status + timestamps), `variants` (dimensions + paths).
Local Filesystem	Stores original uploads and generated variant files using server-controlled unique names.

Prerequisites and Setup

- Go Version 1.22
- PostgreSQL

- The `migrate` CLI (`golang-migrate`):

```
go install -tags 'postgres' github.com/golang-migrate/migrate/v4/cmd/migrate@latest
```

- Web browser able to open HTML pages

Setup

1. Create a database and user:

```
sudo -u postgres psql -c "CREATE USER imagelab WITH PASSWORD 'imagelabpass';"
sudo -u postgres psql -c "CREATE DATABASE imagelab OWNER imagelab;"
```

2. Copy the environment template and edit it if your credentials differ:

```
cp .envrc.example .envrc
source .envrc
```

3. Apply migrations:

```
migrate -path ./migrations -database "$IMAGELAB_DB_DSN" up
```

4. Fetch dependencies:

```
go mod tidy
```

5. Run the server:

```
go run ./cmd/api -db-dsn="$IMAGELAB_DB_DSN"
```

6. Open the app:

```
http://localhost:4000
```
